

Deep Reinforcement Learning for Competitive Agents in MicroRTS

Architecture, Training, and Tournament Evaluation

Master's thesis • June 23, 2026

Master [120] in Computer Science and Engineering

École polytechnique de Louvain, UCLouvain

Author: Mathis DELSART (NOMA 31302100)

Supervisor: Éric PIETTE

Readers: Q. CAPPART, A. MORENVILLE, B. RONVAL

-  **1. Context & Objective**
→ RTS, MicroRTS, research questions.
-  **2. Background**
→ Reinforcement learning and PPO.
-  **3. State of the Art**
→ Milestones, from games to RTS.
-  **4. Contributions**
→ What this thesis delivers.
-  **5. Evaluation Framework**
→ CoG-style tournament, five metrics.
-  **6. Environment Stack**
→ Extended Java–Python bridge.
-  **7. The UECD Architecture**
→ U-Net + Transformer + attention.
-  **8. Training Recipe**
→ PPO pipeline and ablated mechanisms.
-  **9. Results & Interpretation**
→ Ablations, headline, generalization.
-  **10. Discussion & Conclusion**
→ Takeaways, limits, and outlook.

Part 1 / 10

Context & Objective

1. Context & Objective	1–6
2. Background	7–11
3. State of the Art	12–15
4. Contributions	16–17
5. Evaluation Framework	18–21
6. Environment Stack	22–25
7. The UECD Architecture	26–31
8. Training Recipe	32–34
9. Results & Interpretation	35–51
10. Discussion & Conclusion	52–55



- **Competitive video games** (≥ 2 players):
gather resources, build armies, and fight on
a shared map.
- Two skills at once: **macro-management**
and **micro-management**.
- A benchmark for **real-time sequential
decision-making**, before transferring to
real-world problems.





Combinatorial action space

grows with the number of units.



Real-time

no exhaustive deliberation.



Partial information

the opponent's plan is hidden.



Long horizon

planning over thousands of steps.



Multi-unit control

many units acting at once.



Sparse reward

win/loss only, after a whole game.



Non-stationary

the opponent adapts continuously.



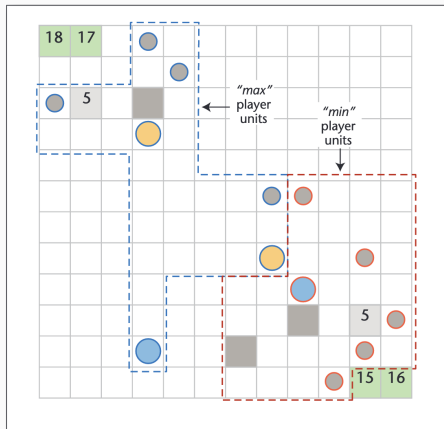
Multi-scale

macro- and micro-management at once.



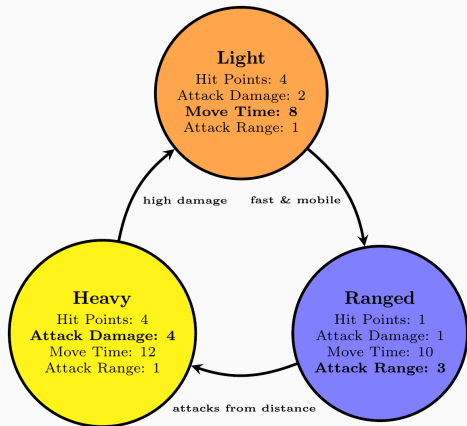
...and many more.

- **AlphaStar** reached Grandmaster in StarCraft II, but at 384 TPUs for 44 days: beyond academic reach.
- **MicroRTS**: a minimal grid-based RTS. Retains the core difficulties, at a fraction of the cost.
- The reference **academic testbed**, with an annual IEEE competition since 2017.



- **Workers:** gather resources and build; fragile (1 HP), enabling early **worker rushes**.
- **Buildings:** Bases produce Workers; Barracks produce military units.
- **Military:** Light, Heavy, and Ranged form a **rock-paper-scissors** cycle.

A strong army must **mix unit types**; a single-type strategy is exploitable.





[gameplay clip – add in PPT]

CoacAI vs RAISocketAI



[gameplay clip – add in PPT]

CoacAI vs RAISocketAI

Real-time, multi-unit battles decided in milliseconds: this is the level a MicroRTS agent must master.



RQ1

Design

Which **architectural and algorithmic design decisions** most improve a MicroRTS agent's performance, and how **robust** is this effect across seeds?



RQ2

Budget

Can an agent competitive with the **strongest prior competition entries** be trained within an **academic compute budget**?

Part 2 / 10

Background

1. Context & Objective	1–6
2. Background	7–11
3. State of the Art	12–15
4. Contributions	16–17
5. Evaluation Framework	18–21
6. Environment Stack	22–25
7. The UECD Architecture	26–31
8. Training Recipe	32–34
9. Results & Interpretation	35–51
10. Discussion & Conclusion	52–55



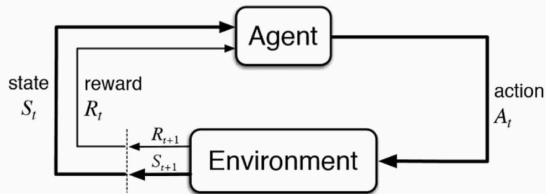
- A recent branch of **machine learning**, rooted in **trial-and-error** learning and **dynamic programming**.
- **Deep** RL: the policy and value functions are deep neural networks.
- Goal: maximize the expected **cumulative reward**.

Find a policy π such that:

$$\max_{\pi} \mathbb{E}_{\pi} [\sum_t \gamma^t R_{t+1}]$$

→ $\gamma \in (0, 1)$: discount factor

→ R_{t+1} : reward of $s_t \rightarrow s_{t+1}$



The agent-environment loop, a **Markov Decision Process**.

Cast as a fully observable **MDP** $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma \rangle$.

$\mathcal{S}$

State: the clock, the resources, and every unit.

→ a combinatorially huge state space.

$\mathcal{A}$

Action: one command per unit, combinatorial and dynamic.

→ an invalid-action mask filters infeasible moves.

$\mathcal{T}$

Transition: the engine's deterministic dynamics.

→ the opponent is folded into the environment.

$\mathcal{R}$

Reward: sparse terminal, $+1/0/-1$ at game end.

→ shaped for training: resources, production, combat.

γ

Discount $\gamma \in (0, 1)$.

→ sets how far ahead the agent plans.

Proximal Policy Optimization



Model-free: learns from play, with no model of the dynamics.

→ unlike model-based, it never simulates the game.



On-policy: trains on fresh data from the current policy.

→ unlike off-policy, it cannot reuse data from past policies.



Actor-critic: a sampled policy $\pi_{\theta}(a | s)$ plus a value critic.

→ unlike value-only or policy-only methods, it combines both.

1. **Policy gradient** push up better-than-average actions

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t \right]$$

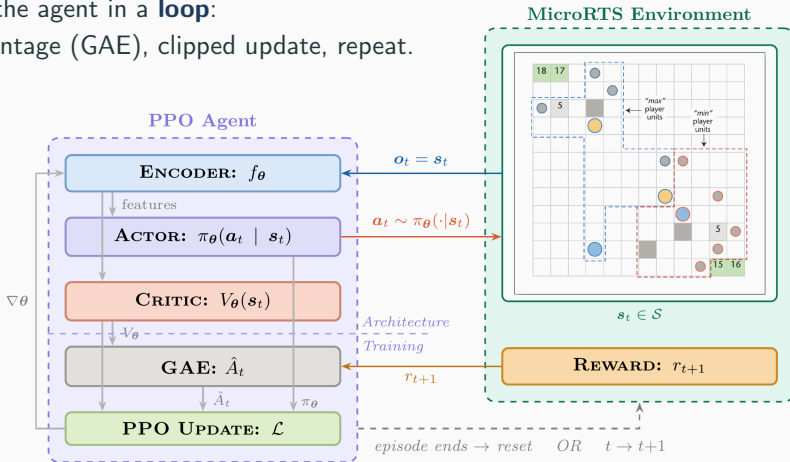
2. **Advantage (GAE)** λ trades bias vs. variance

$$\hat{A}_t^{\text{GAE}} = \sum_{k \geq 0} (\gamma \lambda)^k \delta_{t+k}, \quad \delta_t = R_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

3. **PPO-Clip** a soft trust region, reuse data safely

$$\mathcal{L}^{\text{CLIP}} = \mathbb{E} \left[\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \hat{A}_t) \right], \quad \rho_t = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

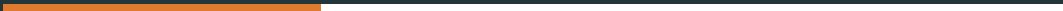
PPO trains the agent in a **loop**:
rollout, advantage (GAE), clipped update, repeat.

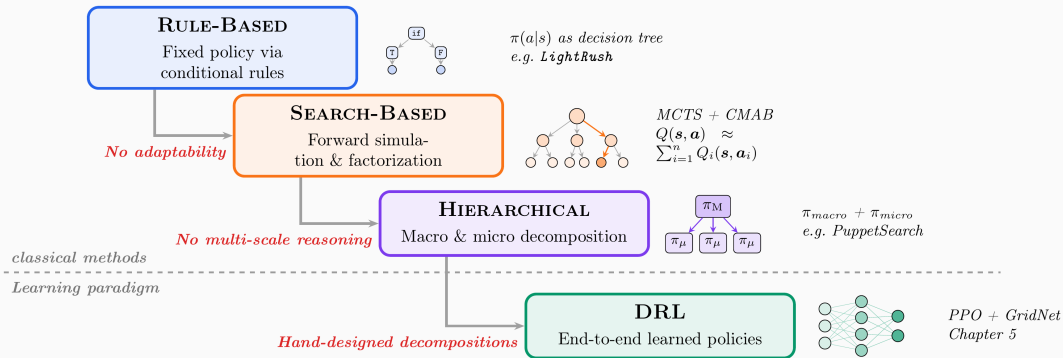


Part 3 / 10

State of the Art

1. Context & Objective	1–6
2. Background	7–11
3. State of the Art	12–15
4. Contributions	16–17
5. Evaluation Framework	18–21
6. Environment Stack	22–25
7. The UECD Architecture	26–31
8. Training Recipe	32–34
9. Results & Interpretation	35–51
10. Discussion & Conclusion	52–55







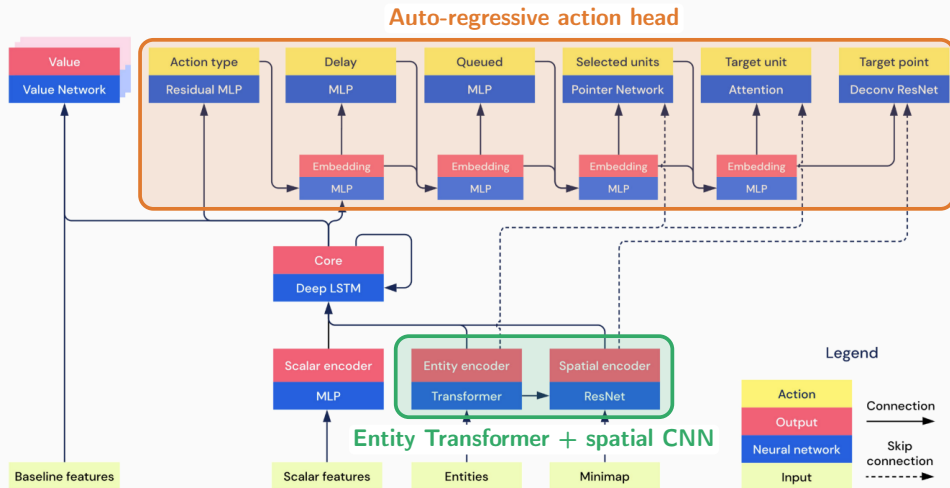
Mastering games: Atari with DQN, then board games with AlphaGo.



Into real-time strategy (2019): AlphaStar in StarCraft II, OpenAI Five in Dota 2.



But at a massive compute cost: AlphaStar ran 384 TPUs for 44 days.



DRL on MicroRTS is still **rare**: most competition entries stay rule- or search-based.

Two agents stand out:

GridNet

Huang et al. – Gym- μ RTS, 2021

Opened up **RL on MicroRTS** with its bridge.

~89% in ~60 GPU-hours vs a **simple 2021 bot pool**; **single-map**, Player 0 only.

RAISocketAI

Goodfriend – CoG, 2023

The **first DRL agent to win** the competition.

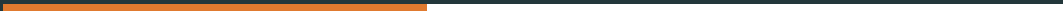
DoubleCone + model portfolio; ~72% win rate; ~23.6 GPU-days on $\leq 16 \times 16$.

This thesis: extend the GridNet line and beat the **whole field** (RAISocketAI as the reference).

Part 4 / 10

Contributions

1. Context & Objective	1–6
2. Background	7–11
3. State of the Art	12–15
4. Contributions	16–17
5. Evaluation Framework	18–21
6. Environment Stack	22–25
7. The UECD Architecture	26–31
8. Training Recipe	32–34
9. Results & Interpretation	35–51
10. Discussion & Conclusion	52–55





1. Tournament framework

→ CoG-style: 12 maps, 15 agents.



2. Environment stack

→ Extended modular Java–Python.



3. UECD architecture

→ U-Net + Transformer + attention.



4. PPO pipeline

→ Flag-gated, ablated in isolation.



5. Rush-collapse analysis

→ Formal, discount-induced collapse.



6. UECD-Best

→ The final competitive agent.

Also an extensive **state-of-the-art survey** (classical and DRL approaches, the benchmark agents), and two **complementary experiments** (behaviour-cloning warm-start, multi-map training).

Combining Spatial and Entity-Based Reasoning for Competitive MicroRTS via U-Net and Transformers

Mathis Delsart
ICTEAM, UCLouvain
Louvain-la-Neuve, Belgium

Achille Morenville
ICTEAM, UCLouvain
Louvain-la-Neuve, Belgium

Éric Piette
ICTEAM, UCLouvain
Louvain-la-Neuve, Belgium

Abstract—In MicroRTS, a real-time strategy benchmark, winning requires long-range coordination between many units. Yet, current deep reinforcement learning agents cannot self-organize spatial features, merge and treat units holistically through stacks of channels. The contributions of this paper are: (i) **MicroRTS**, a hybrid architecture coupling spatial and per-unit reasoning explicitly via a multi-scale convolutional backbone and a Transformer over unit embeddings; (ii) a PPO training regime distilled from systematic ablation; (iii) a reproducible round-robin tournament. On *OpenSpiel*/MicroRTS, UBCD reaches a 58.67% win rate, ahead of all prior competition winners. **Index Terms**—Deep Reinforcement Learning, Real-Time Strategy Games, MicroRTS, Entity-Based Reasoning, Spatial Reasoning, U-Net, Transformers

1. INTRODUCTION

Real-time strategy (RTS) games combine large combinatorial action spaces, multi-unit coordination, sparse long-horizon rewards, and real-time constraints, making them a challenging benchmark for sequential decision-making [1]–[3]. AlphaStar [4] demonstrated Grandmaster-level play in StarCraft II through deep reinforcement learning (DRL), but at an extreme compute cost (3M TPUs over 48 days). MicroRTS [5] distills RTS gameplay into small grid maps while preserving its core difficulties and has served as the IEEE Conference on Games (CoG) competition benchmark since 2017 [6]. Current MicroRTS agents fall into two paradigms: DRL agents such as RAISectorAI [6], dominant on small maps (256x128) where per-unit inference cost remains manageable; and rule-based bots such as CuckAI, Moqut, OnTheBorderline, and TMA, which generalize to larger layouts within tight time budgets.

Purely convolutional backbones capture long-range unit interactions only indirectly. This paper addresses this limitation through: (i) **UBCD**, a hybrid architecture combining a multi-scale encoder-decoder with skip connections (U-Net [7]) gated by channel and spatial attention (CBAM, Convolutional Block Attention Module [8]) for spatial reasoning, and an entity Transformer [9] for unit-level reasoning; and (ii) a **training regime** derived from abating 12 Personal Policy Optimization (PPO) [10] additions, including: (i) **Minichap** (Competitiveness Weighting (MCW) and two-phase opponent-pool fine-tuning), a third contributor; (ii) a **reproducible round-robin tournament**, is motivated by the 2020 CoG MicroRTS edition's shift to Large Language Model (LLM) agents.

On the standard *OpenSpiel*/MicroRTS map, UBCD tops a 1D-agent tournament (just CoG winners, backbones, and 4 UBCD

checkpoints) with a 56.07% overall win-rate (WR) and a 9-1 head-to-head record against RAISectorAI.

II. BACKGROUND AND RELATED WORK

MicroRTS Py [11] exposes MicroRTS as an RL environment. Its *GridStar* PPO baseline, using per-unit action logs with invalid-action masking [12], reaches 35% WR on *OpenSpiel*/MicroRTS as Player 1 (P1) against a 2021-era pod. Building on this framework, RAISectorAI [6], the strongest published DRL agent for MicroRTS, unites a 5M-parameter backbone via self-play and a final opponent pod; several design choices inform the ablations in Section V.

AlphaStar [4] introduced two ideas adopted here: *per-unit* Transformer encoding, which represents units as discrete entities rather than as pixels in a feature map; and *Proximal Excessive Self-Play* (PESP), which samples opponents from a historical league weighted by matchup difficulty.

Further details on competition rules, mechanics, and the environment are available in [3], [11] and the project [Wiki](#).

III. UBCD ARCHITECTURE

Fig. 1 shows the proposed UBCD architecture, short for *U-Net-Encoder-CBAM-Decoder*. Unlike purely spatial backbones, UBCD combines three complementary reasoning axes: (i) **spatial**, (ii) **relational**, and (iii) **global**. (i) **Spatial perception**: A CBAM-gated U-Net [7], [8] backbone (Fig. 1, *Enc* and *Dec*) performs multi-scale spatial reasoning through skip connections between the encoder and decoder. Unlike channel-only Sparse-and-Distillation (SD) attention [13], CBAM jointly weights channels (e.g. merged unit and HP channels during combat) and spatial regions (e.g. frontlines and resource fields). Most depth sits at the bottleneck, where small feature maps keep it cheap.

(ii) **Relational reasoning**: A parallel Entity Transformer [9] (Fig. 1, *grnt*) captures long-range unit interactions explicitly (e.g. units constituting an attack). Each unit is encoded as a token combining raw attributes, spatial features, and positional embeddings. Refined entity features are self-supervised additively back into the spatial map at both bottleneck (strategic) and mid-resolution (tactical) scales.

(iii) **Global spatial reasoning**: A lightweight 4-head self-attention layer over the bottleneck grid (Fig. 1, *net*) captures long-range region-to-region dependencies beyond local convolutional receptive fields (e.g. trading a frontal push against pressure on the home base).

“Combining Spatial and Entity-Based Reasoning for Competitive MicroRTS via U-Net and Transformers”

M. Delsart, A. Morenville, É. Piette – ICTEAM, UCLouvain

The core of this thesis, condensed into a paper submitted to the **IEEE Conference on Games (CoG) 2026**.



Under review.

Decision pending.

Part 5 / 10

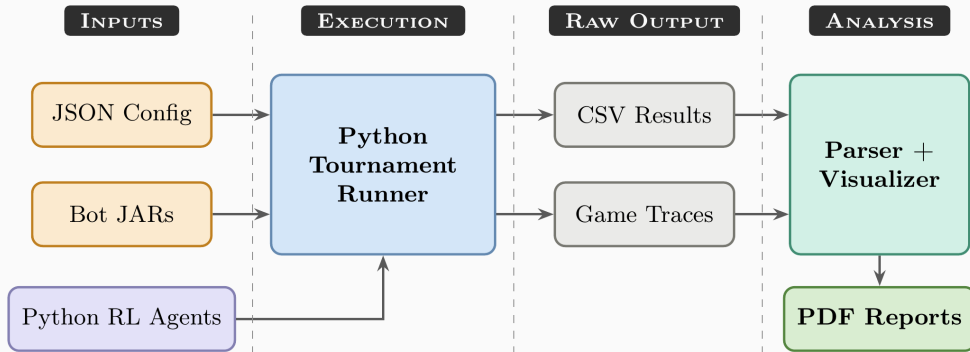
Evaluation Framework

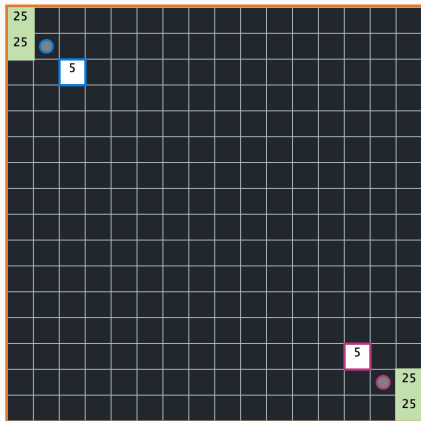
1. Context & Objective	1–6
2. Background	7–11
3. State of the Art	12–15
4. Contributions	16–17
5. Evaluation Framework	18–21
6. Environment Stack	22–25
7. The UECD Architecture	26–31
8. Training Recipe	32–34
9. Results & Interpretation	35–51
10. Discussion & Conclusion	52–55



🏆 **MicroRTS 2026 competition** → 🗨️ **shifted to LLM agents**

so the classic bot tournament is reproduced here, anchored on the 2023 edition.





basesWorkers16x16A

- The IEEE-CoG 2023 edition ships 12 maps (8 open plus 4 held-out, 8x8 to 64x64).
- This thesis concentrates on the standard **basesWorkers16x16A**, the competition's reference map.
- The other maps come back later, shown as they are used (generalization and multi-map experiments).

Built-in baselines (4) a controlled difficulty floor

RandomBiasedAI, WorkerRush, LightRush, NaiveMCTS

Competition winners (11), 7 editions (2017–2024)

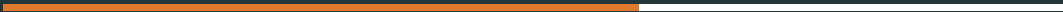
- **Hierarchical:** StrategyTactics, Tiamat, UtsImass, MixedBot.
- **Search:** Droplet.
- **Rule-based:** CoacAI, Mayari, ObiBotKenobi, TMA.
- **DRL:** **GridNet** (the baseline I extend), **RAISocketAI** (2023 champion, my reference).

Metric	Perspective	Question
Win rate	Aggregate	How often does it win overall?
Nash averaging	Worst-case robustness	Payoff vs the optimal adversarial mix?
α -Rank	Evolutionary stability	Can it resist invasion and dominate?
Copeland score	Pairwise dominance	How many opponents beaten head-to-head?
Regret	Robustness gap	How far from the best counter, per opponent?

Part 6 / 10

Environment Stack

1. Context & Objective	1–6
2. Background	7–11
3. State of the Art	12–15
4. Contributions	16–17
5. Evaluation Framework	18–21
6. Environment Stack	22–25
7. The UECD Architecture	26–31
8. Training Recipe	32–34
9. Results & Interpretation	35–51
10. Discussion & Conclusion	52–55



Extended Gym- μ RTS

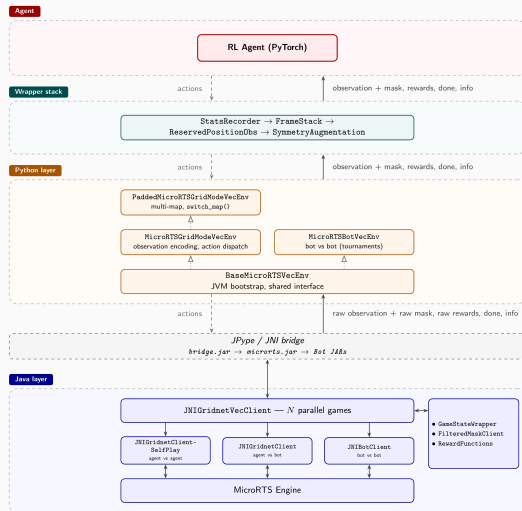
→ substantial Java and Python changes.

Shared JVM (JPyype)

→ one vectorized client steps N games per call.

Three game modes

→ self-play + RL-vs-bot in one batch; bot-vs-bot for evaluation.





- 23 / 55



Invalid-action mask

→ zeroes illegal moves before sampling



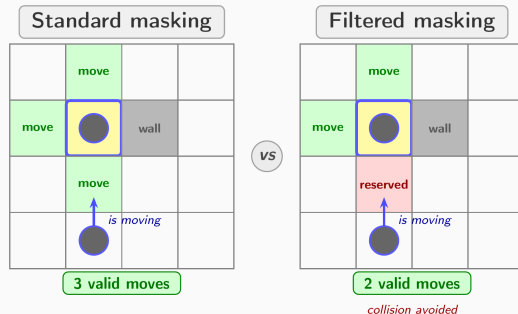
Filtered, destination-aware

→ two units never collide into one cell



Reserved-position channel

→ the agent sees pending conflicts



With the extended observation, the **two biggest feature gains** come from here.

Each is **flag-gated** and switched on independently, on top of the PPO baseline.

Zero padding

- Pads every map to one **fixed-size** tensor
- One network trains across **all map sizes**

Shaped reward

- **1** sparse terminal (Win/Draw/Loss)
- **8** dense event rewards

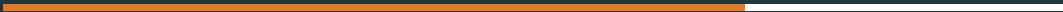
Wrappers

- Frame stacking
- Symmetry augmentation (**4x**)
- Reserved-position obs.

Part 7 / 10

The UECD Architecture

1. Context & Objective	1–6
2. Background	7–11
3. State of the Art	12–15
4. Contributions	16–17
5. Evaluation Framework	18–21
6. Environment Stack	22–25
7. The UECD Architecture	26–31
8. Training Recipe	32–34
9. Results & Interpretation	35–51
10. Discussion & Conclusion	52–55



Each axis motivates **one component** of the final design.

Local

Tactics in a unit's
neighborhood.

→ **Convolutions**

Relational

Unit-to-unit coordination
at any distance.

→ **Transformer**
over unit tokens

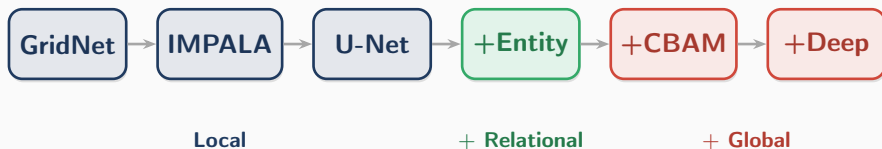
Global

Region-to-region
trade-offs on the map.

→ **Bottleneck**
self-attention

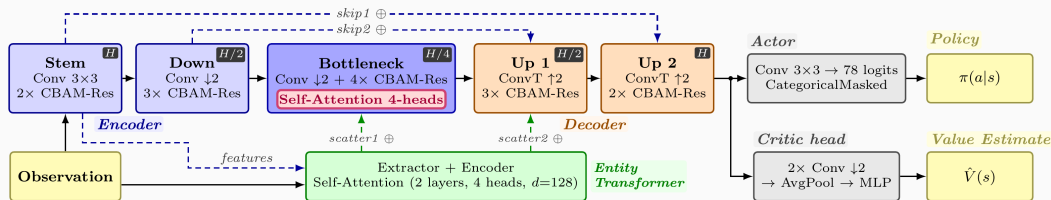
Each step is one incremental, motivated change.

Result: **UECD** = U-Net-Entity-CBAM-Deep.

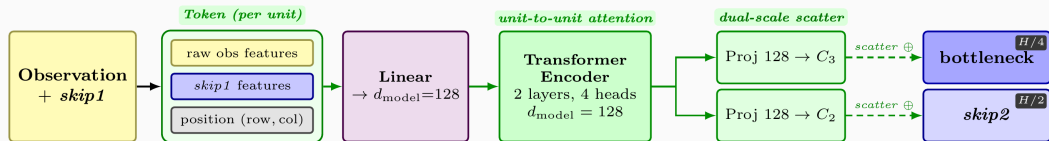


UECD $\rightarrow$ CBAM-gated U-Net + Entity Transformer + bottleneck self-attention

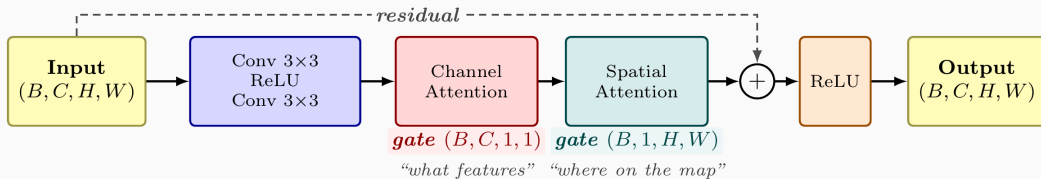
$\sim 4.7\text{M}$ params at $C_1=48$



Per-unit tokens attend to all others (**2 layers, 4 heads, 128-dim**),
scattered back at **two scales**, preserving each unit's identity.



Channel and spatial attention gate each residual block: **what** features and **where** on the map (frontlines, exposed units), at **negligible cost** (under 1% of each block's weights).



Convolutions are local: even on 16×16 , a unit cannot reach the opposite edge.

The **Deep** gives the network a map-wide view.



Bottleneck self-attention

Every region attends to every other in a **single layer**.

Placed at the $H/4 \times W/4$ bottleneck:
 $256 \times$ **cheaper** than full resolution.



Pyramidal depth

Capacity concentrated at the bottleneck;
feeds self-attention and Entity scatter.

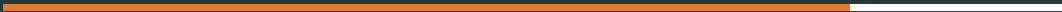


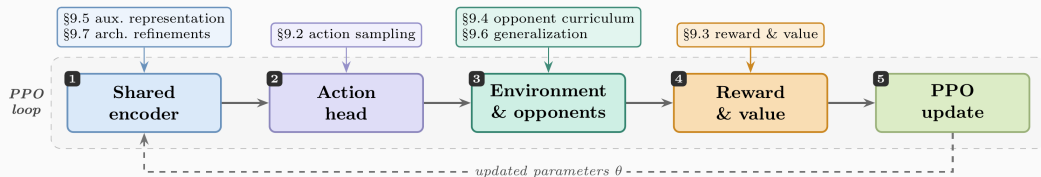
(2, 3, 4, 3, 2)

Part 8 / 10

Training Recipe

1. Context & Objective	1–6
2. Background	7–11
3. State of the Art	12–15
4. Contributions	16–17
5. Evaluation Framework	18–21
6. Environment Stack	22–25
7. The UECD Architecture	26–31
8. Training Recipe	32–34
9. Results & Interpretation	35–51
10. Discussion & Conclusion	52–55





1. Fixed PPO baseline

same hyperparameters; isolates
the variable under study

2. Opt-in mechanisms

each CLI flag-gated, disabled by
default

3. Ablated in isolation

enables the controlled feature
ablation

1. Action sampling

→ autoregressive head, hierarchical sub-action masking

2. Reward & value signal

→ dense-to-sparse scheduling, value heads, PopArt, HL-Gauss, PAE

3. Opponent curriculum & self-play

→ adaptive curriculum, self-play (PFSP), MCW

4. Auxiliary representation

→ four shared-encoder heads: ownership, unit count, contrastive, opponent modeling

5. Generalization

→ symmetry augmentation, multi-map (PLR)

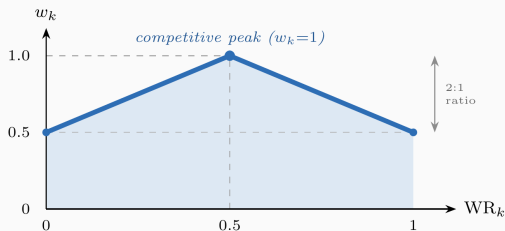
6. Architecture refinements

→ GELU, SPP critic

Around twenty mechanisms, each studied **one at a time**.

Matchup Competitive Weighting: a novel, on-policy reweighting inside the PPO update.

- Each transition is weighted by how **competitive** its matchup is: highest for even ($\sim 50\%$) opponents, lowest for already-decided ones.
- Only the **policy-gradient** term is reweighted; the rollout and the value heads stay untouched.

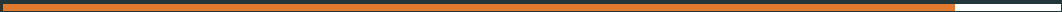


Among the **largest gains** in the ablation.

Part 9 / 10

Results & Interpretation

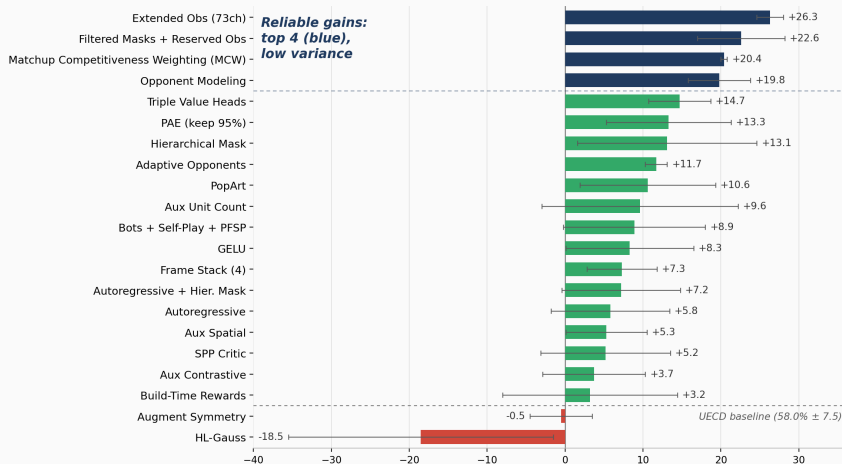
1. Context & Objective	1–6
2. Background	7–11
3. State of the Art	12–15
4. Contributions	16–17
5. Evaluation Framework	18–21
6. Environment Stack	22–25
7. The UECD Architecture	26–31
8. Training Recipe	32–34
9. Results & Interpretation	35–51
10. Discussion & Conclusion	52–55

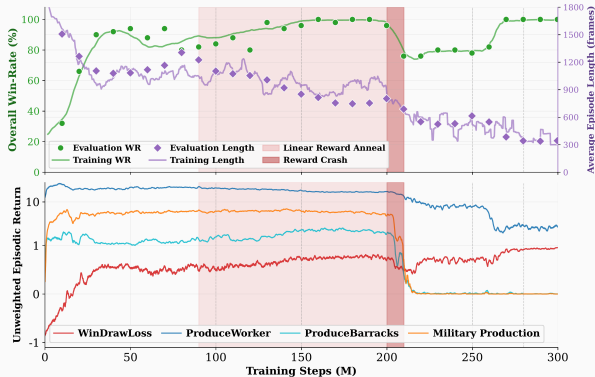


Architecture	Params	Win rate	Std	Δ vs GridNet
<i>Axis 1 – Local spatial</i>				
GridNet (baseline)	838 k	64.8%	± 3.8	—
IMPALA-CNN	1.02 M	68.7%	± 5.8	+3.9 pp
U-Net	2.59 M	77.0%	± 1.6	+12.2 pp
<i>Axis 2 – Long-range relational</i>				
IMPALA-CNN-Entity	1.37 M	74.9%	± 1.2	+10.1 pp
U-Net-Entity	2.90 M	82.7%	± 4.7	+17.9 pp
U-Net-Entity-CBAM	2.90 M	79.1%	± 7.2	+14.3 pp
<i>Axis 3 – Global spatial</i>				
UECD	4.72 M	86.3%	± 7.1	+21.5 pp

7 architectures $\times$ 3 seeds, 100M steps each (13.6 GPU-days); grouped by reasoning axis.

The four reliable gains change **perception** and **gradient allocation**, not capacity (+0.78% params).





- **Setup:** UECD + top-5 scaled to 300M steps, the shaped reward annealed to sparse ($1.0 \rightarrow 0$), with self-play.
- Once shaped $\rightarrow 0$ and $\gamma < 1$, the only way to grow return is a **shorter episode**.
- $750 \rightarrow 300$ frames amplifies the terminal signal $\sim 92\times$: collapse to a worker-rush exploit.



Fix: a 10% shaped-reward floor.

Annealing kills the shaped sum; only the discounted terminal reward survives:

$$J(\pi) \xrightarrow{w_{\text{shape}} \rightarrow 0} w_{\text{term}} \mathbb{E}_{\tau \sim \pi} [\gamma^{T-1} r_T^{\text{term}}] \approx 10 \mathbb{E}_{\tau \sim \pi} [\gamma^{T-1}].$$

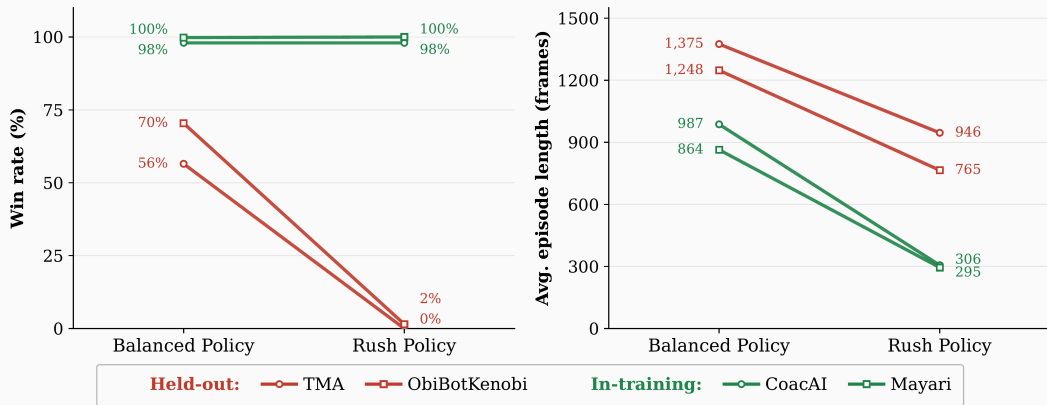
With $\gamma < 1$, γ^{T-1} strictly decreases in T : the only way to grow J is a **shorter episode**, irrespective of play quality.

It reaches the policy through the advantage, and the clip fails to suppress it.
Two rollouts share s_t but end at T and $T - \Delta$ (same state-only $V(s_t)$):

$$\hat{A}_t^{(T-\Delta)} - \hat{A}_t^{(T)} = (\gamma^{-\Delta} - 1) G_t^{(T)} > 0.$$

The shorter rollout always gets the larger advantage;
PPO clips the ratio ρ_t , **not** $\hat{A}_t$, so the gradient flows.

On held-out opponents the rush policy collapses while the balanced one holds: the rush is a **training-distribution exploit**, not transferable.

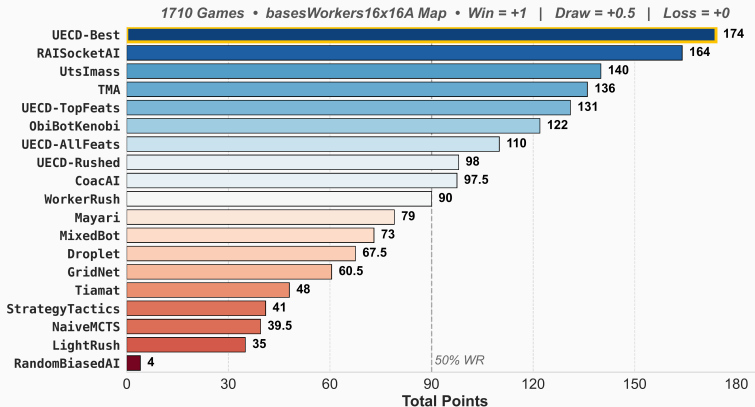


Resume from the **150M (pre-collapse) checkpoint** and fine-tune in two short phases, keeping the 10% shaped floor:

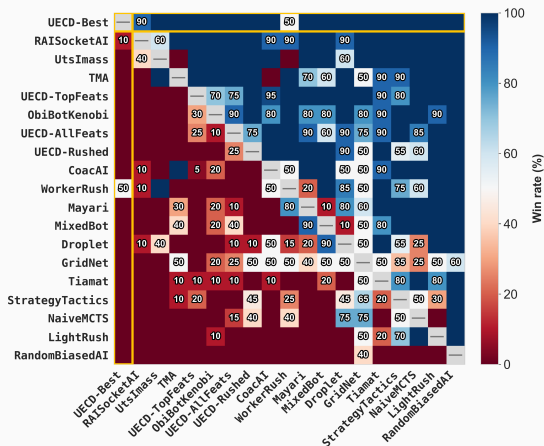
- **Phase 1** (150M $\rightarrow$ 240M): **broaden the pool** — swap the 3 weakest bots for TMA + ObiBotKenobi, more self-play.
- **Phase 2** (240M $\rightarrow$ 350M): **harden** — add RAISocketAI to the opponent pool.

Phase	Steps (M)	On-path	Full run
From scratch	0–150	34.0 h	68.5 h
Phase 1 (broaden pool)	150–240	30.3 h	50.0 h
Phase 2 (vs RAISocketAI)	240–350	108.7 h	108.7 h
Total = UECD-Best	350	7.21 days	9.47 days

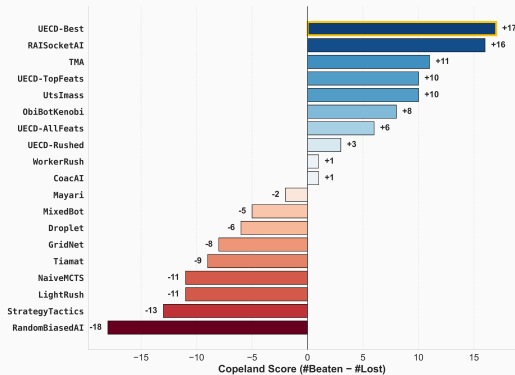
Tournament: UECD-Best tops the field



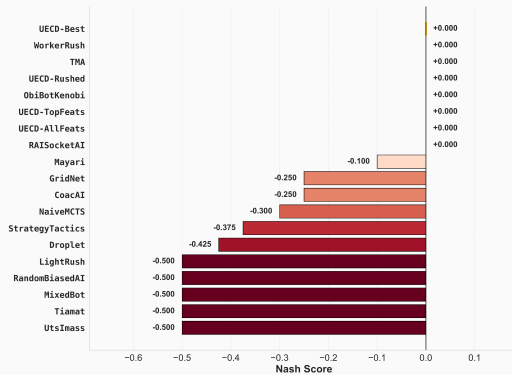
19-agent round-robin on basesWorkers16x16A: 96.67% win rate, first of 19.



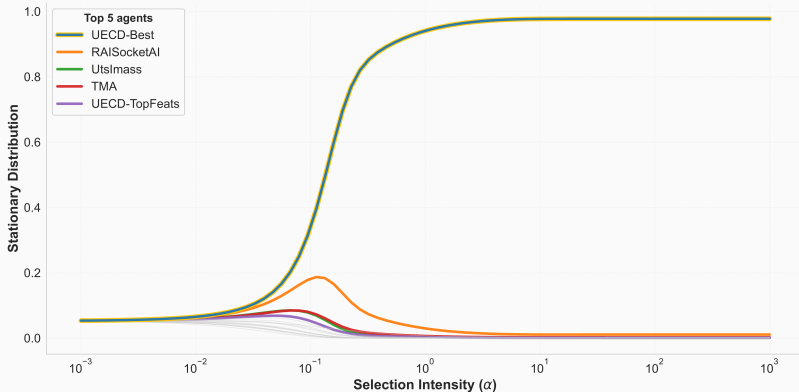
- UECD-Best wins **100%** against every agent but two: WorkerRush (50%) and RAISocketAI (90%).
- This matrix is the CoG protocol: **10 games/matchup**, **deterministic actions** (not sampling).
WorkerRush's 50% is a 2-replay artifact.
- Re-run with **sampling** (1000 games): WorkerRush 99.3%, RAISocketAI **65.7%** (± 2.9). The deterministic 90% is optimistic.



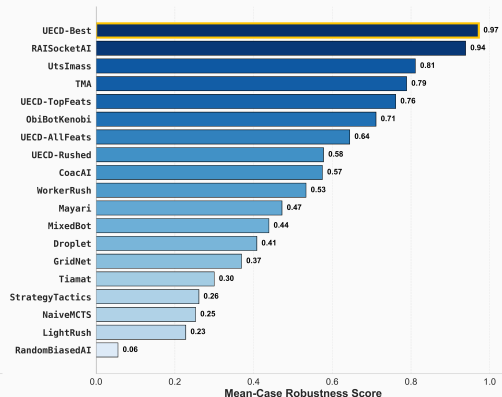
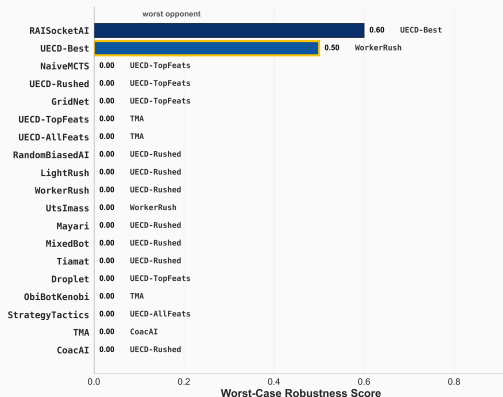
Copeland (1st, +17): breadth — how many opponents you beat, regardless of margin.



Nash (in support): non-exploitable vs the equilibrium mix; immune to padding the pool.

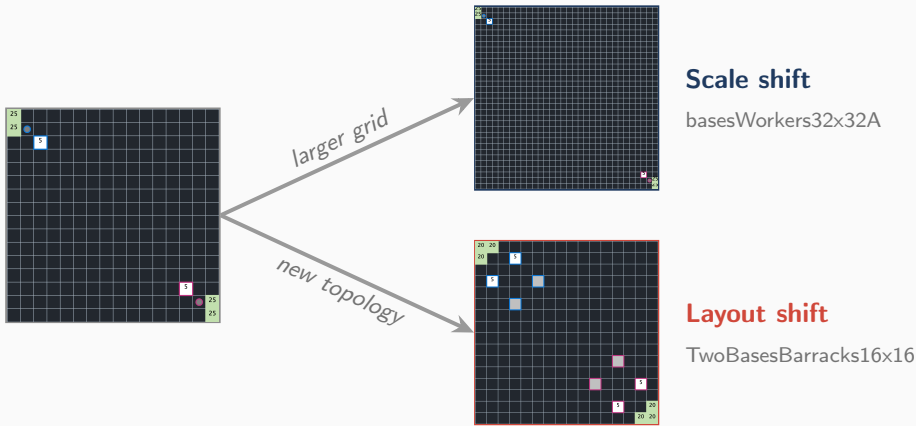


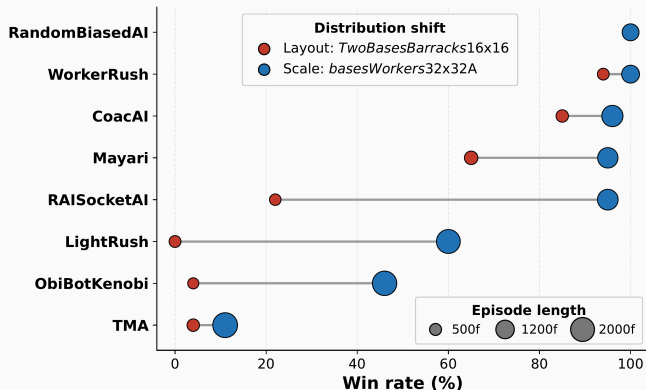
α -Rank (1st): evolutionary stability — the strategy hardest to invade and displace.



Regret (1st avg, 2nd worst-case): **robustness** — the gap to the pool's best counter for each opponent.

Trained **only** on basesWorkers16x16A, then evaluated under two shifts:





Tolerates a **scale** shift (32x32),
but is brittle under a **layout** shift.

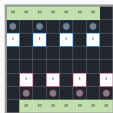
Single-map training overfits
topology more than **size**.

UECD-MultiMap trains on the five open layouts of size $\leq 16 \times 16$, shown here to scale:



8x8

basesWorkers8x8A



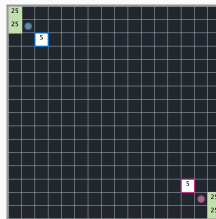
8x8

FourBasesWorkers8x8



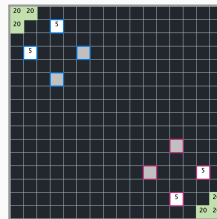
9x8

NoWhereToRun9x8



16x16

basesWorkers16x16A

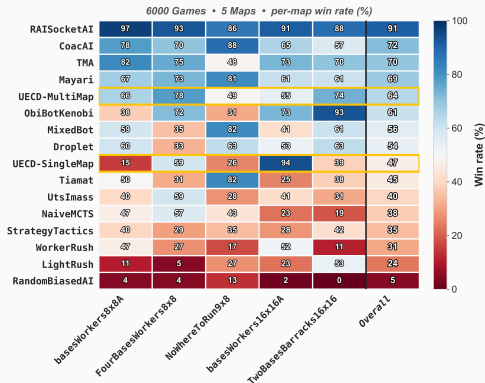


16x16

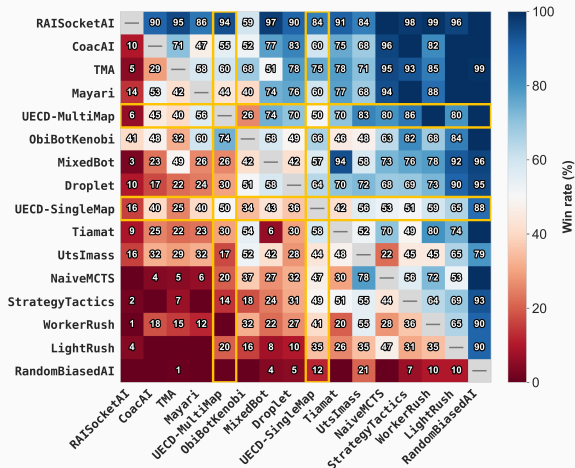
TwoBasesBarracks16x16

Three sizes, five topologies. The small boards are almost a different game:
tiny maps reward an immediate **rush**, unlike the macro play on 16x16.

🎯 **GOAL:** Can a **Prioritized Level Replay (PLR)** curriculum on a **padded environment** train *one* agent across every map size?

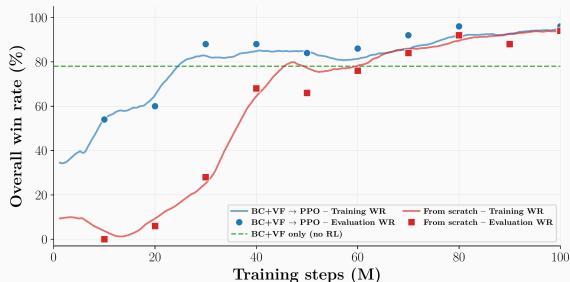


- **UECD-SingleMap collapses on 8x8** (14th/16) - yet 8x8 is only a **scale** shift the probes called tolerable.
- Why: tiny maps reward an immediate **rush**, a strategy padding cannot transfer.
- **UECD-MultiMap** removes the per-map collapse.
- Trade: **82 pts (7th)** vs 141 (1st) at home.



- Head-to-head, **UECD-MultiMap** beats the specialist **UECD-SingleMap** on nearly every matchup.
- Trails only on **RAISocketAI** and **ObiBotKenobi**: the single-map peak still scores higher.
- Only **200M steps / 5 maps**; more budget should flip even those two.

✓ The **PLR + padding** pipeline works: a multi-map agent is feasible.



Warm-start: BC+VF on 300 bot games (~313k transitions)
vs RAISocketAI, CoacAI, Mayari.

Two hypotheses (100M steps):

H1. BC speeds up early PPO? ✓ Yes

88% vs 28% at 30M; +41 pp avg over the first 50M (BC-only already 78%).

H2. Higher final ceiling? ⚖ Inconclusive

96% vs 94% at 100M, within ± 6 pp noise; both plateau at 84–96%.

⚡ BC-PPO buys compute-efficiency, not a higher plateau:
a much stronger agent sooner under a tight budget.

Part 10 / 10

Discussion & Conclusion

1. Context & Objective	1–6
2. Background	7–11
3. State of the Art	12–15
4. Contributions	16–17
5. Evaluation Framework	18–21
6. Environment Stack	22–25
7. The UECD Architecture	26–31
8. Training Recipe	32–34
9. Results & Interpretation	35–51
10. Discussion & Conclusion	52–55

Where I differ

- **Signal beats capacity**: the reliable gains reshape perception and gradient, at near-zero parameter cost.
- **BC** speeds up early PPO but lifts no ceiling on a single map (vs RAISocketAI's multi-map 72%→88%).
- The **rush collapse** has no MicroRTS precedent: formalized, quantified ($\sim 92\times$), and mitigated.

Where I confirm

- **Relational attention** helps multi-unit control, as in StarCraft II agents.
- Single-map training **overfits topology** more than scale.
- **Multi-metric** evaluation surfaces structure a plain win rate hides.

Deliberate scope

- MicroRTS, not a commercial RTS.
- Full observability (no fog-of-war).
- Pure RL: no planner, search, or LLM.
- Final agent is **single-map**; headline from a single run (no seed band).

Open gaps

- A monotone **Ranged-only** strategy (narrow RAISocketAI margin).
- **Topology-shift collapse**: a new layout hurts far more than a larger size.
- Multi-map agent not yet **competition-grade**.



1. Hybrid DRL + LLM

LLM plans and models opponents; the 2026 competition pivoted to LLM agents.



2. Hierarchical policy

Slow strategic tick over per-unit micro-policies.



3. Neuro-symbolic

Compact rules induced from replays gate the policy.



4. Adversarial PCG

A regret-based map generator co-evolves with the agent.

Each **prototypes on the existing codebase** and closes a named limitation.
The long-term vision targets a “**Ludii for RTS**” with shared transfer protocols.



RQ1

Design

The biggest, most **robust** gains come from **perception** (extended obs, masks), **competitive self-play** (MCW), and a **relational + multi-scale** architecture (**UECD**).



RQ2

Budget

Yes: UECD-Best surpasses RAISocketAI on a fixed layout at ~ 9.5 GPU-days, at the cost of cross-layout robustness.

An **open-source pipeline**: a substrate for future hybrid DRL/LLM MicroRTS agents.

Slide	Figure	Source
1	StarCraft II screenshot	Blizzard Entertainment (2010)
1	Warcraft III screenshot	Blizzard Entertainment (2002)
3	MicroRTS game state	Ontañón et al., <i>AI Magazine</i> (2018)
6	Agent–environment loop	Sutton & Barto, <i>RL: An Introduction</i> (2018)
14	AlphaStar architecture	Vinyals et al., <i>Nature</i> (2019)
<i>all</i>	Interface icons	Font Awesome 5 Free (CC BY 4.0)

All other figures and diagrams are the author's own work. All sources are clickable.

Thank you

Questions?



mathis.delsart@gmail.com



github.com/mathisdelsart/microrts-drl-uecd



mathisdelsart.github.io/microrts-drl-uecd-website